

Que. Write a C program to simulate the following CPU scheduling algorithm.

(a) FCFS

(b) SJF

(a) FCFS non-preemptive

```
#include <stdio.h>
```

```
int main () {
```

```
    int n, i;
```

```
    int at[20], bt[20], ct[20], tat[20], wt[20], rt[20];
```

```
    float avg-wt = 0; avg-tat = 0; avg-rt = 0;
```

```
    printf ("Enter arrival Time and Burst time: \n");
```

```
    for (i = 0; i < n; i++) {
```

```
        printf ("P. No. and BT: ", i+1);
```

```
        scanf ("%d", &bt[i]);
```

```
    }
```

```
    ct[0] = at[0] + bt[0];
```

```
    for (i = 1; i < n; i++) {
```

```
        if (ct[i-1] < at[i])
```

```
            ct[i] = at[i] + bt[i]
```

```
        else
```

```
            ct[i] = ct[i-1] + bt[i];
```

```
    }
```

```
    for (i = 0; i < n; i++) {
```

```
        tat[i] = ct[i] - at[i];
```

```
        wt[i] = tat[i] - bt[i];
```

```
        rt[i] = wt[i];
```

avg-wt += wt[i];
 avg-tot += tot[i];
 avg-st += st[i];

```

3
printf("\n Process \t at \t bt \t ct \t rt \t wt \t kt \t n");
for (i=0; i<n; i++) {
  printf("P%d \t %d \t %d \t %d \t %d \t %d \t %d\n",
    i+1, at[i], bt[i], ct[i], tot[i], wt[i], rt[i]);
  3
  printf("\n Average waiting time : %.2f", avg-wt/n);
  printf("\n Average Turn around time : %.2f", avg-tot/n);
  printf("\n Average Response Time : %.2f\n", avg-rt/n);
  return 0;
}

```

O/P

Enter number of process : 3
 Enter Arrival time and burst time :

P1 AT : 0
 BT : 5
 P2 AT : 1

P3 BT : 3

P3 AT : 2

P3 BT : 4

Process	AT	BT	CT	TAT	WT	RT
P1	0	5	5	5	0	0
P2	1	3	8	7	4	4
P3	2	4	12	10	6	6

waiting time = 3.33
 Average Turnaround time = 7.33
 Average Response Time = 3.33

(b) SJF (non-preemptive)

* include <stdio.h>
 * include <limits.h>

```

int main () {
  int n, i, time = 0, completed = 0;
  int at[100], bt[20], ct[20], wt[20], rt[20];
  int done[20] = {0};
  float avg-wt = 0, avg-tot = 0, avg-rt = 0;
  printf("Enter number of processes");
  scanf("%d", &n);
  printf("Enter Arrival time and burst time : \n");
  for (i=0; i<n; i++) {
    printf("P%d AT: ", i+1);
    scanf("%d", &at[i]);
    printf("P%d BT: ", i+1);
    scanf("%d", &bt[i]);
  }
  while (completed < n) {
    int shortest = -1;
    int min-bt = INT_MAX;
    for (i=0; i<n; i++) {
      if (at[i] <= time & done[i] == 0 & bt[i] < min-bt) {
        min-bt = bt[i];
        shortest = i;
      }
    }
    if (shortest == -1) {
      time++;
      continue;
    }
    time = time - at[shortest];
  }
}

```

```

time += bt[shortest];
ct[shortest] = time;
tot[shortest] = ct[shortest] - at[shortest]
wt[shortest] = tot[shortest] - bt[shortest]

```

```
done[shortest] = 1;
```

```
completed;
```

```
avg-wt += tot[shortest];
```

```
avg-tot += tot[shortest];
```

```
avg-rt += t[shortest];
```

```
}
```

```
printf("\n Process P1, P2, P3 and wt P3 \n");
```

```
for (i=0; i<n; i++) {
```

```
printf("%d %d %d %d %d %d %d",
```

```
i+1, at[i], bt[i], ct[i], tot[i], wt[i],
```

```
rt[i]);
```

```
}
```

```
printf("\n Average waiting time = 1.24, avg-wt = 1.24,
```

```
printf("\n Average turnaround time = 1.24, avg-tot = 1.24,
```

```
printf("\n Average Response Time = 1.24, avg-rt = 1.24,
```

```
)
```

o/p Enter number of process = 3

Enter Arrival time and burst time:

P1	P2	P3
AT: 0	AT: 1	AT: 2
BT: 5	BT: 3	BT: 4

Process	P1	P2	P3
AT	0	1	2
BT	5	3	4
CT	5	8	12
TAT	5	7	10
WT	0	4	6
RT	0	4	6

Average waiting time = 3.33

Average turn Around Time = 7.33

Average Response time = 3.33

• SJF (pre-emptive)

```
# include <stdio.h>
```

```
# include <i
```

```
int main() {
```

```
int n, i, time = 0, completed = 0;
```

```
int at[20], bt[20], rem_bt[20];
```

```
int ct[20], tot[20], wt[20], rt[20];
```

```
int first[20] = {0};
```

```
float avg-wt = 0, avg-tot = 0, avg-rt = 0;
```

```
printf("Enter number of processes: ");
```

```
scanf("%d", &n);
```

```
printf("Enter arrival time and burst time: \n");
```

```
for (i=0; i<n; i++) {
```

```
printf("%d %d AT: ", i+1,
```

```
scanf("%d %d", &at[i], &bt[i]);
```

```
printf("%d %d BT: ", i+1,
```

```
scanf("%d %d", &at[i], &bt[i]);
```

```
rem_bt[i] = bt[i];
```

```
rt[i] = -1;
```

```
}
```

```
while (completed < n) {
```

```
int shortest = -1;
```

```
int min_bt = INT_MAX;
```

```

    }
    if (shortest == -1) {
        time++;
        continue;
    }
    if (rt[shortest] == -1)
        rt[shortest] = time - at[shortest];
    rem_bt[shortest]--;
    time++;
    if (rem_bt[shortest] == 0) {
        completed++;
        Ct[shortest] = time;
        tat[shortest] = Ct[shortest] - at[shortest];
        wt[shortest] = tat[shortest] - bt[shortest];
        avg_wt += wt[shortest];
        avg_tat += tat[shortest];
        avg_rt += rt[shortest];
    }
    printf("\n Process P1 B1 CT TAT WT RT\n");
    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
            i+1, at[i], bt[i], Ct[i], tat[i], wt[i]);
    }
    printf("\n Average Waiting Time = %.2f", avg_wt/n);
    printf("\n Average Turnaround time = %.2f", avg_tat/n);
    printf("\n Average Response Time = %.2f", avg_rt/n);
}

```

Output:

Enter number of process: 3
Enter Arrival time and Burst time:

P1 P1: 0
P1 P1: 5

P2 P2: 1
P2 P2: 3

P3 P3: 2
P3 P3: 4

Process	AT	BT	CT	TAT	WT	RT
P1	0	5	8	8	3	0
P2	1	3	4	3	0	0
P3	2	4	12	10	6	6

Average waiting time = 3.00

Average Turnaround time = 7.00

Average Response Time = 2.00

Ch 13/3/26

Lab 2: Write a C program to simulate schedule Algorithm to find TAT, WOT, priority (preemptive & non-preemptive).

Program Code:

preemptive:

```
#include <stdio.h>
#include <limits.h>

int main()
```

```
{
    int n, i, time = 0, completed = 0;
```

```
    int at[20], bt[20], pr[20], rem_bt[20];
```

```
    int ct[20], tat[20], wt[20], rt[20];
```

```
    float avg_wt = 0, avg_tat = 0, avg_rt = 0;
```

```
    printf("Enter no. of processes: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter Arrival Time, burst time and priority: \n");
```

```
    for (i = 0; i < n; i++)
```

```
{
    printf("P%d AT: ", i+1);
```

```
    scanf("%d", &at[i]);
```

```
    printf("P%d BT: ", i+1);
```

```
    scanf("%d", &bt[i]);
```

```
    printf("P%d Priority: ", i+1);
```

```
    scanf("%d", &pr[i]);
```

```
    rem_bt[i] = bt[i];
```

```
    rt[i] = -1;
```

```
}
while (completed < n)
```

```
{
```

```
    int highest = -1;
```

```
    int min_pr = INT_MAX;
```

```
    for (i = 0; i < n; i++) {
```

```
        if (at[i] <= time && sum_bt[i] > 0 && pr[i] == min_pr)
            highest = i;
```

```
    }
```

```
    if (idx != -1)
```

```
    {
        start[idx] = time;
```

```
        res[idx] = start[idx] - at[idx];
```

```
        printf("P%d ", i+1);
```

```
        time += bt[idx];
```

```
        ct[idx] = time;
```

```
        tat[idx] = ct[idx] - at[idx];
```

```
        wt[idx] = tat[idx] - bt[idx];
```

```
        visited[idx] = 1;
```

```
        completed++;
```

```
    }
```

```
    time++;
```

```
    }
```

```
    while (completed < n)
```

```
    {
        int idx = -1;
```

```
        int high = 0;
```

```
        for (i = 0; i < n; i++)
```

```
        {
            if (at[i] <= time && rt[i] > 0)
```

```
            {
                if (pr[i] < high)
```

```
                {
                    high = pr[i];
```

```
                    idx = i;
```

```
                }
            }
        }
```

```
        if (idx != -1)
```

```
        {
            start[idx] = -1;
```

```
            stop[idx] = time;
```

```
            res[idx] = stop[idx] - at[idx];
```

```
            printf("P%d ", i+1);
```

```
            rt[idx]--;
```

Smatt chart : $P_2 | P_1 | P_3 |$

	P1D	AT	B1	R	CT	TAT	CT	RT
P1	1	4	3	5	4	0	0	
P2	0	1	2	1	1	0	3	
B3	2	6	1	11	9	3		

2 - 1

3 0

3 0

9

log Turnaround Time = 4.67

Avg waiting time = 1.00

~~Aug~~ ~~pet and~~ time = 1.00

~~Aug waiting time = 1.00~~
~~time = 1.00~~

Fig Turnaround Time = 4.67

3 2 0 1 11

p_2	0	1	2	1
0	0	0	0	0
1	0	0	0	0
2	0	0	0	0
1	0	0	0	0

1
2
3
4

PID AT BT PR CT

Small chart: $P_2 | P_1, P_3 |$

Round Robin:

#include <stdio.h>

* define MAX 100

typedef struct {

int pid, arrived, burst;

int remaining, completion, turnaround, waiting;

int start-time, response;

} Process;

typedef struct {

int data [MAX];

int front, rear;

} Queue;

void initQueue (Queue * q) {

q->front = q->rear = -1;

}

int isEmpty (Queue * q) {

return q->front == -1;

}

void enqueue (Queue * q, int val) {

if (q->rear == MAX-1) return;

if (q->front == -1) q->front = 0;

q->data [q->rear] = val;

}

int dequeue (Queue * q) {

int val = q->data [q->front];

if (q->front == q->rear) return;

q->front = q->rear + 1;

else

q->front++;

return val;

void sortByArrival (Process p[], int n) {

Process temp;

for (int i = 0; i < n; i++) {

if (p[i].Arrival > p[i+1].Arrival) {

temp = p[i];

p[i] = p[i+1];

p[i+1] = temp;

}

}

int main () {

int n, tq;

Process p [MAX];

Queue q;

initQueue (&q);

printf ("Enter number of processes: ");

scanf ("%d", &n);

for (int i = 0; i < n; i++) {

printf ("\n Process %d \n", i+1);

printf ("PID: ");

scanf ("%d", &p[i].pid);

printf ("Arrival time: ");

scanf ("%d", &p[i].arrival);

printf ("Burst time: ");

scanf ("%d", &p[i].burst);

p[i].remaining = p[i].burst;

p[i].start-time = -1;

}

```

Scanf ("%d", &ta);
SortByArrival (p, n);
int time = 0, completed = 0, i = 0;
int gantt [MAX * 10], gindex = 0;
enqueue (&q, 0);
i = 1;
while (completed < n) {
    if (isEmpty (&q)) {
        time++;
        while (i < n && p[i].arrival <= time) {
            enqueue (&q, i);
            i++;
        }
        continue;
    }
    current = dequeue (&q);
    if (p[current].arrival > time)
        time = p[current].arrival;
    if (p[current].start_time == -1) {
        p[current].start_time = time;
        p[current].start_time += time;
        p[current].response = time - p[current].arrival;
    }
    int current - time = (p[current].remaining < ta)?
    :
    for (int k = 0; k < exec_time; k++) {
        gantt [gindex++] = p[current].pid;
        time++;
    }
}

```

```

while (i < n && p[i].arrival <= time) {
    enqueue (&q, i);
    i++;
}
p[current].remaining -= exec_time;
if (p[current].remaining == 0) {
    p[current].completion = time;
    p[current].turnaround = time - p[current].arrival;
    p[current].waiting = p[current].turnaround - p[current].burst;
    completed++;
} else {
    enqueue (&q, current);
}
}
float totalwaitWT = 0, totalTAT = 0, totalRT = 0;
printf ("n n ID \t AT \t BT \t CT \t TAT \t WT \t RT \t n");
for (int i = 0; i < n; i++) {
    printf ("%d \t %d \t %d \t %d \t %d \t %d \t %d \t %d \t n",
        p[i].pid, p[i].arrival, p[i].burst, p[i].completion,
        p[i].turnaround, p[i].waiting, p[i].response);
    totalWT += p[i].waiting;
    totalTAT += p[i].turnaround;
    totalRT += p[i].response;
}
printf ("n Average WT = %.2f", totalWT / n);
printf ("n Average TAT = %.2f", totalTAT / n);
printf ("n Average RT = %.2f", totalRT / n);

```

```

print ("\n Grant Credit");
for (int i = 0; i < gIndex; i++) {
    printf ("%P.i.d | " + grant [i]);
}

printf ("\n 0");
for (int i = 1; i <= gIndex; i++) {
    printf ("%P.i.d", i);
}

return 0;
}

```

Output 1:

Enter no. of processes:

Process 1
PID : 123
Arrival time : 3
Burst time : 4

Process 2
PID : 124
Arrival time : 2
Burst time : 5

Process 3
PID : 125
Arrival time : 2
Burst time : 4

Enter time Quantum : 3

PID	AT	BT	CT	TAT	WT	RT
124	2	5	10	8	3	0
125	2	4	11	8	4	2
123	3	4	11	8	4	2

Average WT = 3.50
Average TAT = 8.00
Average RT = 1.00

Grant Credit

P124 | P124 | P124 | P123 | P123 | P124 | P123
0 1 2 3 4 5 6 7 8 9

Output 2:

Enter no. of processes : 2

PID : 1
Arrival time : 3
Burst time : 4

PID : 2
Arrival time : 2
Burst time : 5

Enter time Quantum : 5

PID	AT	BT	CT	TAT	WT	RT
2	2	5	7	5	0	0
1	3	4	11	8	4	4

Average WT = 2.00
Average TAT = 6.50
Average RT = 2.00

Grant Credit:

P2 | P2 | P2 | P2 | P1 | P1 | P1 | P1
0 1 2 3 4 5 6 7 8 9

Conclusion:

- Small TQ = more switching, large TQ = less switching
- Small TQ improved responsiveness

• Multilevel queue:

* include <stdio.h>

* define MAX 100

typedef struct {

int pid, arrival, burst;

int remaining, completion, turnaround, waiting;

int start-time, response;

} process;

typedef struct {

int data [MAX];

int front, rear;

} queue;

void initQueue (Queue * q) {

q->front = q->rear = -1;

}

int isEmpty (Queue * q) {

return q->front == -1;

}

void enqueue (Queue * q, int val) {

if (q->rear == MAX-1) return;

if (q->front == -1) q->front = 0;

q->data [q->rear] = val;

}

void dequeue (Queue * q) {

int val = q->data [q->front];

if (q->front == q->rear)

q->front = q->rear = -1;

else

q->front++;

return val;

}

void SortByArrival (Process p[], int n) {

Process temp;

for (int i = 0; i < n-1; i++) {

for (int j = i+1; j < n; j++) {

if (p[i].arrival > p[j].arrival) {

temp = p[i];

p[i] = p[j];

p[j] = temp;

}

}

int main () {

int n;

Process p[MAX];

Queue System Q, user Q;

initQueue (&system Q);

int queue (&user Q);

printf ("Enter no. of processes: ");

scanf ("%d", &n);

for (int i = 0; i < n; i++) {

printf ("\n Process %d\n", i+1);

printf ("PID: ");

scanf ("%d", &p[i].pid);

printf ("Arrival time: ");

scanf ("%d", &p[i].arrival);

printf ("Burst time: ");

scanf ("%d", &p[i].burst);

```

printf (" \n Gantt Chart : \n \n ");
for (int i = 0; i < gindex; i++) {
    if (gantt[i] == -1)
        printf ("Idle \n");
    else
        printf ("P%d |", gantt[i]);
}
printf ("\n0")
for (int i = 1; i < gindex; i++) {
    printf (" | %d", i);
}
}
printf (" \n");
return 0;
}

```

Output:

Enter no. of processes: 4

Process 0

Enter arrival time: 0

Enter burst time: 5

Enter type (0 = system, 1 = user): 0

Process 1

Enter arrival time: 2

Enter burst time: 3

Enter type (0 = system, 1 = user): 1

Process 2

Enter arrival time: 1

Enter burst time: 4

Enter type (0 = system, 1 = user): 0

Process 3

Enter arrival time: 3

Enter burst time: 2

Enter type (0 = system, 1 = user): 1

ITD	type	AT	BT	CT	WT	TAT
0	system	0	5	5	0	5
1	user	2	3	12	7	10
2	system	1	4	9	4	8
3	user	3	2	14	9	11

Avg. waiting time: 5.

Avg. turnaround Time: 8.5

Round Robin:

Process 1

PID: 1

Arrival time: 3

Burst time: 4

Process 2

PID: 2

Arrival time: 2

Burst time: 5

Process 3:

PID: 3

Arrival time: 2

Burst time: 4

Enter time Quantum: 3

PID	AT	BT	CT	TAT	WT	RT
1	3	4	15	12	8	8
2	2	5	13	11	6	0
3	2	4	14	12	8	3

Avg WT = 7.33

Avg TAT = 11.67

Avg RT = 2.67

~~Grant Credit~~

~~(P2) (P2) (P3) (P3) (P1) (P1) (P1) (P2) (P3) (P1)~~
~~0 1 2 3 4 5 6 7 8 9 10 11 12~~

Process 1

~~PID: 1~~

~~Arrival time: 3~~

~~Burst~~

Enter time Quantum: 5

PID	AT	BT	CT	TAT	WT	RT
1	3	4	15	12	8	8
2	2	5	7	5	0	0
3	2	4	11	9	5	5

Avg WT = 4.33

Avg TAT = 8.67

Avg RT = 4.33

~~Grant Credit~~

~~(P2) (P2) (P2) (P2) (P3) (P3) (P1) (P1) (P1)~~

~~0 1 2 3 4 5 6 7 8 9 10 11 12~~

Program

→ write program for 1) Rate monotonic

2) EDF

3) Proportional Share

```
#include <math.h>
```

```
#include <stdio.h>
```

```
#define MAX_P 20
```

```
typedef struct
```

```
{
```

```
int id;
```

```
int burst;
```

```
int deadline;
```

```
int period;
```

```
int cT;
```

```
int ut;
```

```
int tot;
```

```
} process;
```

```
void copy-processes (process dst[], process src[], int n)
```

```
{
```

```
int i;
```

```
for (i=0; i<n; i++)
```

```
dst[i] = src[i];
```

```
}
```

```
double Equ-utilization (process p[], int n)
```

```
{
```

```
int i;
```

```
double u = 0.0;
```

```
for (i=0; i<n; i++)
```

```
{
```

```
u += (double) p[i].burst / p[i].period;
```

```
}
```

```
return u;
```

```
void compute-ct-ut-tot (process p[], int n)
```

```
{
```

```
int i;
```

```
int time = 0;
```

```
for (i=0; i<n; i++)
```

```
{
```

```
time += p[i].burst;
```

```
p[i].ct = time;
```

```
p[i].tot = p[i].ct;
```

```
p[i].ut = p[i].ct;
```

```
}
```

```
}
```

```
void sort-by-deadline (process p[], int n)
```

```
{
```

```
int i, j;
```

```
for (i=0; i<n-1; i++)
```

```
{
```

```
if (p[i].deadline > p[i+1].deadline) {
```

```
process temp = p[i];
```

```
p[i] = p[i+1];
```

```
p[i+1] = temp;
```

```
}
```

```
}
```

```
void sort-by-period (process p[], int n)
```

```
{
```

```
int i, j;
```

```
for (i=0; i<n-1; i++)
```

```
{
```

```
for (j=i+1; j<n; j++)
```

```
{
```

```
if (p[i].period > p[j].period)
```

```
{
```

```
process temp = p[i];
```

```
p[i] = p[j];
```

```
p[j] = temp;
```

```
}
```

```
}
```



```

scanf("%d", &P.T.T.Deadline);
printf("Period (for) &ns: ");
scanf("%d", &P.T.T.Period);
printf("n");

```

```

Run - ed1(P.n)
Print - ms(P.n);
Print - Proportioned(P.n);

```

Out put:
 n. process : 3
 Process 0 :
 burst time : 3
 deadline : 7
 period : 20
 weight : 5
 Process 1 :
 burst time : 2
 deadline : 4
 period : 5
 weight : 7
 Process 2 :
 burst : 2
 deadline : 8
 period : 10
 weight : 23

CPU utilisation : 0.75

ID	BT	D	CT	WT	TAT
0	2	4	2	0	2
1	3	7	5	2	5
2	2	8	7	5	7

RMS scheduling
 CPU utilisation : 0.75

Band : 0.780

ID	BT	P	CT	WT	TAT
0	2	5	2	0	2
1	2	10	4	2	4
2	3	20	7	4	7

Proportional Schedule

ID	BT	CPU Share
0	3	42.857%
1	2	28.571%
2	2	28.571%

28.571%
 28.571%
 28.571%

Program 5

Bounded Buffer:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
```

```
#include <unistd.h>
```

```
#define BUFFER_SIZE 3
```

```
#define ITEMS 15
```

```
int buffon [BUFFER_SIZE];
```

```
int in = 0;
```

```
int out = 0;
```

```
pthread_mutex_t mutex;
```

```
sem_t empty;
```

```
sem_t full;
```

```
void producer (void *arg) {
```

```
for (int i = 0; i < ITEMS; i++) {
```

```
sem_wait(&empty);
```

```
pthread_mutex_lock(&mutex);
```

```
buffon[in] = i;
```

```
printf("Produced: %d at buffon [%d] \n", i, in);
```

```
in = (in + 1) % BUFFER_SIZE;
```

```
pthread_mutex_unlock(&mutex);
```

```
sem_post(&full);
```

```
usleep(100000);
```

```
}
```

```
return NULL;
```

```
}
```

```
void *consumer (void *arg) {
```

```
for (int i = 0; i < ITEMS; i++)
```

```
{ sem_wait(&full);
```

```
pthread_mutex_lock(&mutex);
```

```
int item = buffon[out];
```

```
printf("Consumed: %d from buffon [%d] \n", item, out);
```

```
out = (out + 1) % BUFFER_SIZE;
```

```
pthread_mutex_unlock(&mutex);
```

```
sem_post(&empty);
```

```
usleep(15000);
```

```
}
```

```
return NULL;
```

```
}
```

```
int main () {
```

```
pthread_t prod, con;
```

```
pthread_mutex_init(&mutex, NULL);
```

```
empty = sem_open("/empty", O_CREAT, 0644, BUFFER_SIZE);
```

```
full = sem_open("/full", O_CREAT, 0644, 0);
```

```
pthread_create(&prod, NULL, producer, NULL);
```

```
pthread_create(&con, NULL, consumer, NULL);
```

```
pthread_join(prod, NULL);
```

```
pthread_join(con, NULL);
```

```
sem_close(empty);
```

```
sem_close(full);
```

```
sem_unlink("/empty");
```

```
sem_unlink("/full");
```

```
pthread_mutex_destroy(&mutex);
```

```
return 0;
```

```
}
```

Output :

```
produced : 0 at buffer[0]
consumed : 0 from "
produced : 1 at buffer[1]
consumed : 1 from buffer[1]
produced : 2 at buffer[2]
consumed : 2 from buffer[2]
produced : 3 at buffer[3]
produced : 4 .. [4]
```

consumed : 14 from buffer[14]

• Dining Philosopher problem :

```
* include <stdio.h>
* include <stdlib.h>
* include <pthread.h>
* include <unistd.h>

#define N 5

pthread_mutex_t forks[N];

void *philosopher(void *arg)
```

```
{
    int id = *(int *) arg;
    int left = id;
    int right = (id + 1) % N;
```

while (1)

```
{
    printf ("Philosopher %d is thinking.\n", id);
```

usleep (10000);

if (id % 2 == 0)

```
{
    pthread_mutex_lock (& forks [left]);
```

```
    printf ("Philosopher %d picked up fork %d.\n", id, left);
```

```
    pthread_mutex_lock (& forks [right]);
```

```
    printf ("Philosopher %d picked up fork %d.\n", id, right);
```

```
}
    printf ("Philosopher %d is eating.\n", id);
    usleep (15000);
```

```
    pthread_mutex_unlock (& forks [left]);
```

```
    pthread_mutex_unlock (& forks [right]);
```

```
    printf ("Philosopher %d put down forks %d and %d.\n", id, left, right);
}
```

```
int main()
```

```
{
    pthread_t threads[N];
    int i;
    for (int i = 0; i < N; i++)
```

```
{
    pthread_mutex_init(&mutex[i], NULL);
```

```
for (int i = 0; i < N; i++)
```

```
{
    pthread_create(&threads[i], NULL, philosopher, &id[i]);
```

```
for (int i = 0; i < N; i++)
```

```
{
    id[i] = i;
```

```
pthread_create(&threads[i], NULL, philosopher, &id[i]);
```

```
for (int i = 0; i < N; i++)
```

```
{
    pthread_join(threads[i], NULL);
```

```
return 0;
```

Output: philosopher 0 is thinking
philosopher 1 is thinking
philosopher 2 is thinking

philosopher 3 is thinking
philosopher 4 is thinking

philosopher 5 is thinking
philosopher 6 is thinking
philosopher 7 is thinking
philosopher 8 is eating

• Code 2 tracing

fork 0 b/w p0 & p1
fork 1 b/w p0 & p1
fork 2 b/w p1 & p2
fork 3 b/w p2 & p3
fork 4 b/w p3 & p4

p0, p1, p2, p3, p4 → thinking

p1 → right → 2

p1 → left → 1

p1 → eat

p3 → right → 4

p3 → left → 3

p3 → eat

p4 → right → 0

p4 → left → 4 → busy → wait

• Code 1 tracing:

[---]

Producer → in

Consumer → out



[0] [---] → [0---]

Consumer 0 [0---] → [---]

buffer → full → produce → wait

buffer → empty → consume → wait

2m/10

• Banker's algo:

#include <stdio.h>

int main() {

int n, m, i, j, k;

printf("Enter no. of processes: ");

scanf("%d", &n);

printf("Enter no. of resource types: ");

scanf("%d", &m);

int alloc[n][m], max[n][m], need[n][m];

int avail[m];

printf("Enter Allocation Matrix: \n");

for (i=0; i<n; i++) {

for (j=0; j<m; j++) {

scanf("%d", &alloc[i][j]);

}

printf("Enter maximum matrix: \n");

for (i=0; i<n; i++) {

for (j=0; j<m; j++) {

scanf("%d", &max[i][j]);

}

printf("Enter Available resources: \n");

for (i=0; i<m; i++) {

scanf("%d", &avail[i]);

}

for (i=0; i<n; i++) {

for (j=0; j<m; j++) {

need[i][j] = max[i][j] - alloc[i][j];

}

int finish[n], safeSeq[n];

for (i=0; i<n; i++)

finish[i] = 0;

int work[m];

for (i=0; i<m; i++)

work[i] = avail[i];

int count = 0;

while (count < n) {

int found = 0;

for (i=0; i<n; i++) {

if (finish[i] == 0) {

for (j=0; j<m; j++) {

if (need[i][j] > work[j])

break;

}

if (j == m) {

for (k=0; k<m; k++)

work[k] += alloc[i][k];

safeSeq[count++] = i;

finish[i] = 1;

found = 1;

}

```

    print ("in system in state : 'n'");
    return 0;
}

```

Enter available resources:

3 3 2

System is in SAFE state;

Safe Sequence :

P1 P3 P4 P0 P2

```

    )
    print ("in system is in SAFE state : 'n'");
    for (i=0; i<n; i++)
        print ("P1.d", safeSeq[i]);

```

```

    print ("n");
    return 0;
}

```

Output:

Enter no. of processes : 5
 " " " resource types : 3

Enter Allocation matrix

0	1	0
2	0	0
3	0	2
2	1	1
0	0	2

Enter maximum matrix:

7	5	3
3	2	2
9	0	2
2	2	2
4	3	3

• Deadlock detection :

* include <stdio.h>

int n, m, i, j, k;

printf ("Enter number of processes : ");

scanf ("%d", &n);

printf ("Enter no. of resource types : ");

scanf ("%d", &m);

int alloc [n][m], request [n][m], avail [m];

printf ("Enter Allocation matrix : 'n'");

for (i=0; i<n; i++)

for (j=0; j<m; j++)

scanf ("%d", &alloc[i][j]);

}

printf ("Enter Request matrix : 'n'");

for (i=0; i<n; i++)

for (j=0; j<m; j++)

scanf ("%d", &request[i][j]);

}

```

for (i=0; i < n; i++)
    scanf("%d", &avail[i]);

```

```

    int finish[n];

```

```

    for (i=0; i < n; i++)
        finish[i] = 0;

```

```

    int work[n];

```

```

    for (i=0; i < m; i++)
        work[i] = avail[i];

```

```

    int found = 0;

```

```

    do {

```

```

        found = 0;

```

```

        for (i=0; i < n; i++)

```

```

            if (finish[i] == 0)

```

```

                for (j=0; j < m; j++)

```

```

                    if (request[i][j] > work[j])

```

```

                        break;

```

```

                }

```

```

                if (j == m)

```

```

                    for (k=0; k < m; k++)
                        work[k] += avail[i];

```

```

                    finish[i] = 1;

```

```

                    found = 1;

```

```

            }

```

```

        }

```

```

    } while (found == 0);

```

```

    int deadlock = 0;

```

```

    for (i=0; i < n; i++)

```

```

        if (finish[i] == 0)

```

```

            deadlock = 1;

```

```

            printf("Process %d is deadlocked", i);

```

```

        }

```

```

    }

```

```

    if (deadlock == 0)

```

```

        printf("No deadlock detected\n");

```

```

    return 0;

```

```

}

```

Output

Processes : 5

Resources : 3

Allocation matrix :

```

0 1 0

```

```

2 0 0

```

```

2 0 2

```

```

2 1 1

```

```

0 0 2

```

Request Matrix :

```

0 0 0

```

```

2 0 2

```

```

0 0 0

```

```

1 0 0

```

```

0 0 2

```

Deadlocked Processes: None

Processes: 3
Resources: 3

Allocation Matrix:

1 0 0

0 1 0

0 0 1

Request Matrix:

0 1 0

0 0 1

1 0 0

Available Resources:

0 0 0

Deadlocked process: ~~P0, P1, P2~~

• Program 7: Memory allocation

* Include <stdio.h>

void firstFit (int blockSize[], int blocks, int processSize,

int processes)

{
int allocation [50];

int i, j;

for (i=0; i < processes; i++)

allocation[i] = -1;

for (i=0; i < processes; i++)

{
for (j=0; j < blocks; j++)

{
if (blockSize[j] >= processSize[i])

{
allocation[i] = j;

blockSize[j] -= processSize[i];

break;

}

printf ("1st fit -- 1st j");

printf ("Process no. \t Process size \t Block no.\n");

for (i=0; i < processes; i++)

{
printf ("%d \t %d \t %d", i+1, processSize[i],

if (allocation[i] != -1)

printf ("%d \t %d", allocation[i] + 1);

else

printf ("Not Allocated\n");

void bestFit (int blockSize[], int blocks, int processSize,

int processes)

{
int allocation [50];

int i, j, bestIdx;

```
for (i = 0; i < processes; i++)
    allocation[i] = -1;
```

```
for (i = 0; i < processes; i++)
```

```
    bestIdx = -1;
```

```
    for (j = 0; j < blocks; j++)
```

```
        if (blockSize[j] >= processSize[i])
```

```
            if (bestIdx == -1 || blockSize[j] < allocation[bestIdx])
                bestIdx = j;
```

```
    }
```

```
    if (bestIdx != -1)
        allocation[i] = bestIdx;
        blockSize[bestIdx] -= processSize[i];
    }
```

```
}
```

```
printf("\n ---Worst Fit ---\n");
printf("Process no. \t Process size \t Block no. \n");
```

```
for (i = 0; i < processes; i++)
```

```
    printf("Process no. \t Process size \t Block no. \n");
```

```
    if (allocation[i] == -1)
```

```
        printf("Process no. \t Process size \t Block no. \n");
```

```
    else
```

```
        printf("Not allocated\n");
```

```
main()
```

```
{
    int blocks, processes, i;
```

```
    int blockSize[50], processSize[50];
```

```
    printf("Enter no. of memory blocks: ");
```

```
    scanf("%d", &blocks);
```

```
    printf("Enter size of %d memory blocks: \n", blocks);
```

```
    for (i = 0; i < blocks; i++)
```

```
        scanf("%d", &blockSize[i]);
```

```
        firstBlock[i] = blockSize[i];
```

```
        bestBlock[i] = -1;
```

```
    }
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &processes);
```

```
    printf("Enter size of %d processes: \n", processes);
```

```
    for (i = 0; i < processes; i++)
```

```
        scanf("%d", &processSize[i]);
```

```
    }
```

```
    printf("Process no. \t Process size \t Block no. \n");
```

```
    for (i = 0; i < processes; i++)
```

```
        printf("Process no. \t Process size \t Block no. \n");
```

```
        if (allocation[i] == -1)
```

```
            else
```

Output:
 Enter Sizes of 5 memory blocks:
 100 500 200 300 600

Enter sizes of 4 processes:

212 417 112 426

First fit

Process No.	Process Size	Block No.
1	212	2
2	417	5
3	112	2
4	426	Not allocated

Best fit

Process No.	Process Size	Block No.
1	212	4
2	417	2
3	112	3
4	426	5

Worst fit

Process No.	Process Size	Block No.
1	212	5
2	417	2
3	112	5
4	426	Not allocated

Que. Given 6 memory allocation of 300Kb, 600Kb, 350Kb, 250Kb, 750Kb, 125Kb
 Size of processes: 115Kb, 600Kb, 358Kb, 200, 335Kb

First fit

Process No.	Process Size	Block No.
1	115	1
2	600	2
3	358	5
4	200	3
5	335	5

Best fit

Process No.	Process Size	Block No.
1	115	6
2	600	2
3	350	5
4	200	4
5	335	5

Worst fit

Process No.	Process Size	Block No.
1	115	5
2	600	5
3	358	2
4	200	3
5	335	5

Not allocated

Page Replacement

① FIFO:

* include <stdio.h>

int main() {

int pages[50], frame[10];

int n, f, i, j, k = 0, pagefault = 0;

if found;

printf("Enter number of pages: ");

scanf("%d", &n);

printf("Enter page reference string: \n");

for (i = 0; i < n; i++) {

scanf("%d", &pages[i]);

printf("Enter no. of frames: ");

scanf("%d", &f);

for (i = 0; i < f; i++) {

frame[i] = -1;

printf("\n Page\t Frame \n");

for (i = 0; i < n; i++) {

found = 0;

for (j = 0; j < f; j++) {

if (frame[j] == pages[i]) {

found = 1;

break;

if (found == 0) {

frame[k] = pages[i];

k = (k + 1) % f;

pagefault++;

printf("%d\t", pages[i]);

for (j = 0; j < f; j++) {

if (frame[j] == -1)

printf("%d\t", frame[j]);

else

printf("-\t");

printf("\n");

printf("\n total page faults = %d \n", pagefault);

Output:

Enter number of pages: 12

Enter page reference string:

1 2 3 4 1 2 5 1 2 3 4 5

Enter no. of frames: 3

Page Frame

1 - -

2 1 2 -

3 1 2 3

4 4 2 3

1 4 1 3

2 4 1 2

5 5 1 2

1 5 1 2

2 5 1 2

3 5 3 2

4 5 3 4

5 5 3 4

Total page faults = 9

• LRU

```
<stdio.h>
```

```
int main () {
```

```
    int pages[50], frame[10], time[10];
```

```
    int n, f, i, j, pos, pagefault = 0;
```

```
    int counter = 0, found;
```

```
    printf("Enter no. of pages : ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter page reference string : \n");
```

```
    for (i = 0; i < n; i++)
```

```
        scanf("%d", &pages[i]);
```

```
    printf("Enter no. of frames : ");
```

```
    scanf("%d", &f);
```

```
    for (i = 0; i < f; i++)
```

```
        frame[i] = -1;
```

```
    time[i] = 0;
```

```
    for (i = 0; i < n; i++)
```

```
        found = 0;
```

```
        for (j = 0; j < f; j++)
```

```
            if (frame[j] == pages[i])
```

```
                counter++;
```

```
            time[j] = counter;
```

```
            found = 1;
```

```
            break;
```

```
    if (found == 0)
```

```
        pos = 0;
```

```
        for (j = 1; j < f; j++)
```

```
            pos = j;
```

```
    frame[pos] = pages[i];
```

```
    printf("%d\t", page[i]);
```

```
    for (j = 0; j < f; j++)
```

```
        if (frame[j] == -1)
```

```
            printf("%d", frame[j]);
```

```
        else
```

```
            printf("-");
```

```
    printf("\n");
```

```
    printf("Total Page faults = %d\n", pagefault);
```

```
    return 0;
```

```
    Enter no. of pages : 12
```

```
    Enter page reference string :
```

```
    1 2 3 4 1 2 5 1 2 3 4 5
```

```
    Enter no. of frames : 3
```

```
    Page      Frames
```

```
    1          1 - -
```

```
    2          1 2 -
```

```
    3          1 2 3
```

```
    4          4 2 3
```

```
    1          4 1 3
```

```
    2          4 1 2
```

```
    5          5 1 2
```

```
    1          5 1 2
```

```
    2          5 1 2
```

```
    3          3 1 2
```

```
    4          3 4 2
```

```
    5          3 4 5
```

```
    total page  
    faults = 10
```

• optimal

```
#include <stdio.h>
```

```
int main() {
```

```
    int pages[50], frame[10];
```

```
    int n, f, i, j, k, pos, pageFault = 0;
```

```
    int found, forward, index;
```

```
    printf("Enter no. of pages: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter page reference string:\n");
```

```
    for (i = 0; i < n; i++) {
```

```
        scanf("%d", &pages[i]);
```

```
    }
```

```
    printf("Enter no. of frames: ");
```

```
    scanf("%d", &f);
```

```
    for (i = 0; i < f; i++) {
```

```
        frame[i] = -1;
```

```
    }
```

```
    for (i = 0; i < n; i++) {
```

```
        found = 0;
```

```
        for (j = 0; j < f; j++) {
```

```
            if (frame[j] == pages[i]) {
```

```
                found = 1;
```

```
                break;
```

```
            }
```

```
            if (found == 0; j < f; j++) {
```

```
                frame[j] = pages[i];
```

```
                break;
```

```
            }
```

```
        if (found == 0) {
```

```
            forward = -1;
```

```
            for (j = 0; j < f; j++) {
```

```
                ind = -1
```

```
                for (k = 1 + j; k < n; k++) {
```

```
                    if (frame[k] == pages[i]) {
```

```
                        index = k;
```

```
                    }
```

```
                    if (index == -1) {
```

```
                        pos = j;
```

```
                        break;
```

```
                    }
```

```
                    if (index > forward) {
```

```
                        pos = j;
```

```
                    }
```

```
                }
```

```
                frame[pos] = pages[i];
```

```
                pageFault++;
```

```
                for (j = 0; j < f; j++) {
```

```
                    if (frame[j] != -1) {
```

```
                        printf("%d", frame[j]);
```

```
                    }
```

```
                printf("\n total page faults = %d\n", pageFault);
```

Enter no. of pages = 12

Enter page reference String = 1 2 3 4 1 2 5 1 2 3

Enter no. of frames : 3

Page	Frames
------	--------

1	1 - -
---	-------

2	1 2 -
---	-------

3	1 2 3
---	-------

4	1 2 4
---	-------

1	1 2 4
---	-------

2	1 2 4
---	-------

5	1 2 5
---	-------

1	1 2 5
---	-------

3	1 2 5
---	-------

4	3 2 5
---	-------

5	4 2 5
---	-------

	4 2 5
--	-------

total page fault = 7

~~88~~
2215126